Ira A. Fulton Schools of Engineering
Arizona State University

CSE 531: Distributed and Multiprocessor Operating Systems

CSE 531 gRPC Written Report

Casey Tom

Problem Statement

This project involves building a distributed banking system where multiple customers can perform transactions—deposit, withdraw, and query balance—across multiple bank branches. Each customer connects to a unique branch based on a shared ID, which allows them to access and update account information.

Assumptions:

- All customers share the same bank account.
- Each customer interacts only with a specific branch that shares their unique ID.
- No concurrent updates occur on the bank account, ensuring sequential transaction processing.
- Each branch maintains a balance replica that should remain consistent with other branch replicas.
- Despite independent updates by each customer to a specific branch, every branch's replica should reflect all account changes made by other branches.

Project Goal

The goal of this project is to implement a distributed banking system based on the requirements outlined in the problem statement. Specifically, this system should support customer-to-branch communications, ensure consistent balance updates across branches, and maintain transactional integrity across distributed nodes.

Major Tasks:

1. Implement customer and branch processes using Python's multiprocessing to enable parallel operations.
2. Develop a mechanism to launch branch processes, which can handle withdrawals, deposits, and balance queries.
3. Parse input from a JSON file to initialize branch balances and process customer action requests.
4. Implement request handling for customers, including deposit, withdraw, and query operations.
5. Set up inter-process communication using gRPC for reliable data exchange.
6. Define the interfaces (Query, Deposit, Withdraw) used between customers and branches, and the propagation interfaces (Propagate_Request) used between branches.

Setup

The setup for this project involves a development environment and specific packages that facilitate distributed communication, structured data handling, and process management.

Development Environment:

- **Python 3.12.3**: Main programming language for implementing distributed processes and gRPC communication.
- **IDE: PyCharm Professional Edition**: Used for project development, debugging, and testing.

- **Anaconda 2024.06**: Manages dependencies and provides an isolated environment for project components.

Python Packages:

- **Multiprocessing**: Manages concurrent customer and branch processes to allow parallel transactions.
- **Concurrent.futures**: Provides tools for asynchronous task execution.
- **gRPC (grpcio==1.64.1)**: Enables Remote Procedure Calls between customers and branches, supporting scalable communication.
- **gRPC Tools (grpcio-tools==1.64.1)**: Generates Python code from Protocol Buffers (.proto files), creating necessary gRPC client and server stubs.
- **Protocol Buffers (protobuf==5.27.2)**: Standardizes message serialization, enabling efficient communication between distributed components.
- **sys and json**: Facilitate command-line argument retrieval and JSON parsing, respectively, for reading input and storing output data.

These technologies are selected to ensure compatibility, facilitate data serialization, and support message transmission across distributed components in the banking system

Implementation Processes

The system design leverages gRPC to ensure consistent and synchronized communication between clients (customers) and servers (branches). Each implementation step is structured to maintain data integrity and consistent balance across all branches.

1. **Proto File Definition**:
    - The banks.proto file defines two message types (MsgRequest and MsgResponse) and a Branch service with two RPC methods: MsgDelivery and MsgPropagation.
    - MsgRequest includes fields for transaction ID, operation type (interface), and transaction amount (money), establishing a standard for request messages.
    - MsgResponse defines fields for operation type, transaction result, and balance, ensuring a consistent structure for responses.
    - This setup ensures reliable communication between clients and branches, with well-defined messages for each transaction type.
2. **Python Code Generation Using gRPC Tools**:
    - Using the grpc_tools.protoc command, the .proto file is compiled into Python files (banks_pb2.py for message structures and banks_pb2_grpc.py for RPC framework implementation).
    - These files provide the basis for gRPC client-server interactions, enabling Python-based communication with the MsgRequest and MsgResponse message types and the Branch service's methods.
3. **Server-Side Implementation (Branch)**:
    - The Branch class (defined in branch.py) extends BranchServicer, implementing the MsgDelivery and MsgPropagation methods.

        - MsgDelivery processes deposits, withdrawals, and balance queries, updating the branch's balance and returning request results. In addition, this will maintain consistency in balance replicas by propagating each withdraw and deposit request.
        - MsgPropagation synchronizes transactions for the branch by implementing the deposit or withdraw to maintain consistency in balance.
    - Additional methods:
        - createStubs establishes gRPC channels for inter-branch communication.
        - ProcessRequest handles transaction types and triggers propagation for deposits and withdrawals, using Propagate_Request functions for consistency.
4. **Server Initialization and Startup (Server.py)**:
    - This file manages the setup and initialization of branch servers:
        - It reads branch information from gprc_input.json and uses createBranchProcesses to instantiate branches and start each as a separate process.
        - Each branch process calls branch_serve, where gRPC channels and client stubs are initialized, enabling the branch to process incoming messages.
5. **Client-Side Implementation (Customer)**:
    - The Customer class (in customer.py) defines each customer's communication with a branch, managing transaction requests.
        - createStub sets up the gRPC channel to the specific branch.
        - executeEvents sends transactions (deposit, withdraw, query) from the JSON input file to the branch and logs results in recvMsg.
    - Customers use stubs to send structured requests to branches and store responses for validation.
6. **Client Initialization and Startup (Client.py)**:
    - This file initializes customer processes by reading from gprc_input.json, parsing customer requests, and using createCustomerProcesses to create clients.
    - serveCustomer creates the gRPC stub and executes the customer's requests, logging responses in an output JSON file for result analysis.

Results

The results of the implementation are captured in the output.json file, which logs each customer's transaction request and the resulting balance after each operation. The JSON output provides a clear record of each request—whether it's a deposit, withdrawal, or query—along with the updated balance.
Here's a sample from the JSON output produced after the processes were done:


```
[
  {
    "id": 1,
    "recv": [
      {
        "interface": "deposit",
        "result": "success"
      },
```

```
        {
          "interface": "query",
          "balance": 410
        }
      ]
    },
    {
      "id": 2,
      "recv": [
        {
          "interface": "deposit",
          "result": "success"
        },
        {
          "interface": "query",
          "balance": 420
        }
      ]
    }
]
```


Detailed Analysis of Results:

1. **Deposit and Query Operations**: The output demonstrates that each initial deposit is correctly processed, with the corresponding balance update accurately reflected in subsequent queries. For example, a deposit of 10 units raises the starting balance of 400 to 410, as seen in the sample above.
2. **Withdrawal Handling**: In cases where a customer performs a withdrawal, the output shows a decrease in balance, provided that sufficient funds are available. If the balance is less than the withdrawal amount, the transaction should fail, and the output will show "result": "fail" for that transaction. This ensures that all transactions respect balance constraints, maintaining consistency across branches.
3. **Consistency Across Branches**: The MsgPropagation method ensures that each branch maintains a synchronized replica of the balance. The output JSON reflects consistent balances across branches after every transaction, validating that the system achieves data integrity in a distributed environment.
4. **Justification of Results**:
    - The results validate that the distributed banking system functions as expected, with accurate handling of deposits, withdrawals, and balance queries.
    - The consistent JSON output across multiple transactions and branches serves as evidence of the system's reliability in maintaining up-to-date balances.

In summary, the output.json file confirms that the distributed system performs correctly across varied transaction scenarios, successfully demonstrating a fully functional, consistent, and resilient distributed banking application.